

Travail pratique 3_2

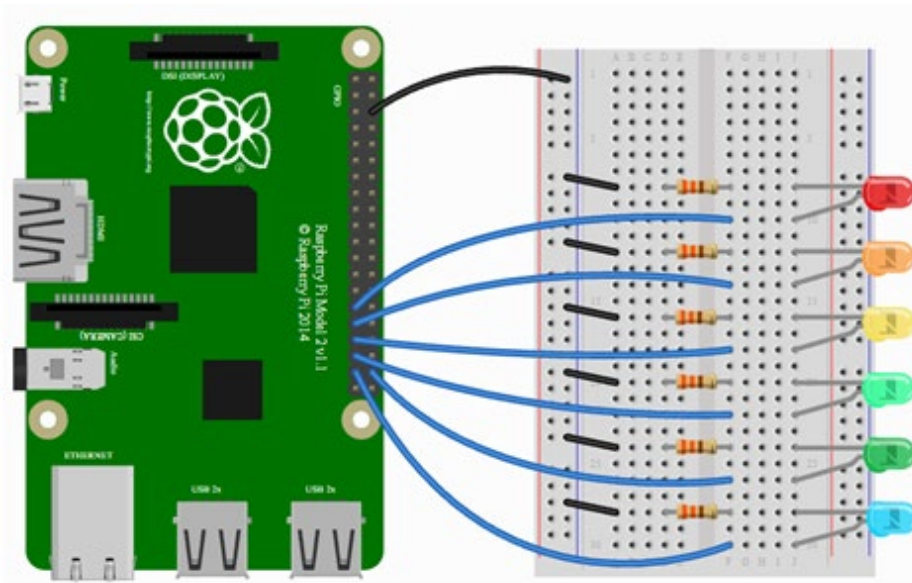
La bibliothèque RPi.GPIO

On s'initiera ici à la bibliothèque RPi.GPIO en convertissant un signal analogique vers un signal numérique. On aura besoin des composants suivantes:

Un Potentiomètre <i>pour fournir une tension variable entre 0 et 3.3 volts à l'entrée du circuit</i>	Un Circuit intégré <i>pour convertir le signal analogique du potentiomètre en une valeur numérique entre 0 et 255</i>	Une rampe de LEDs <i>pour afficher la valeur numérique reçu du convertisseur</i>
 Potentiometer	 ADC Module	 Led Bar Graph

Votre rampe devra allumer autant de LEDS que la valeur de la tension du potentiomètre. Aucune pour zéro volt, dix pour 3.3 volts. Les LEDS qui sont allumées le seront en cascade (voir le clip de [démonstration](#)). Finalement, la vraie valeur numérique, celle issue de votre convertisseur, devra aussi être affiché par votre script Python dans la console. **Ce travail doit faire usage de la bibliothèque RPi.GPIO; il n'est pas permis d'utiliser *gpiozero*, en tout ou en partie.**

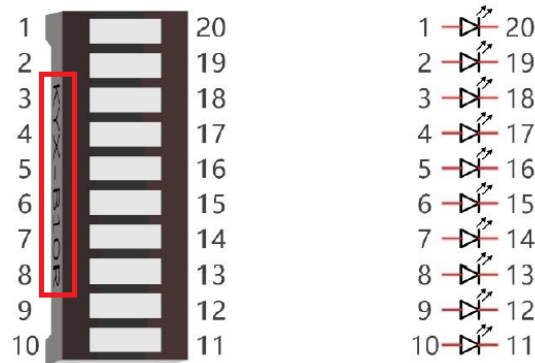
La rampe de LEDS : ce composant est strictement une version compacte de dix LEDS contigües. Prenons par exemple le circuit de test suivant:



En RPi.GPIO, le code de test suivant allumera et éteindra les LEDS rouge et orange:

```
>>> import RPi.GPIO as GPIO
>>> GPIO.setmode(GPIO.BCM) # nomenclature GPIO pour les numéros de broches
>>> GPIO.setup([5,6], GPIO.OUT) # on met les broches en mode sortie
>>> GPIO.output([5,6], GPIO.HIGH) # on les allume (HIGH = 3.3 volts)
>>> GPIO.output([5,6], GPIO.LOW) #on les éteint
>>> GPIO.cleanup() # on fait le ménage
```

Il s'agit pour vous d'adapter ce code pour allumer les bonnes LEDS sur votre rampe. La configuration des broches de la rampe de LEDS est la suivante:



Remarquez que les anodes sont à gauche, les cathodes à droite, en supposant l'étiquette du composant orienté vers gauche (encadré en rouge). Chaque segment est une LED, donc n'oubliez pas vos résistances!

Potentiomètre: ce composant permet de générer une tension variable en faisant tourner la tige rotative. Les broches de GND et VCC (3.3 volts) peuvent être inversées et dicteront le sens de l'augmentation de tension. La broche OUT est la sortie de tension analogique que vous devez convertir.

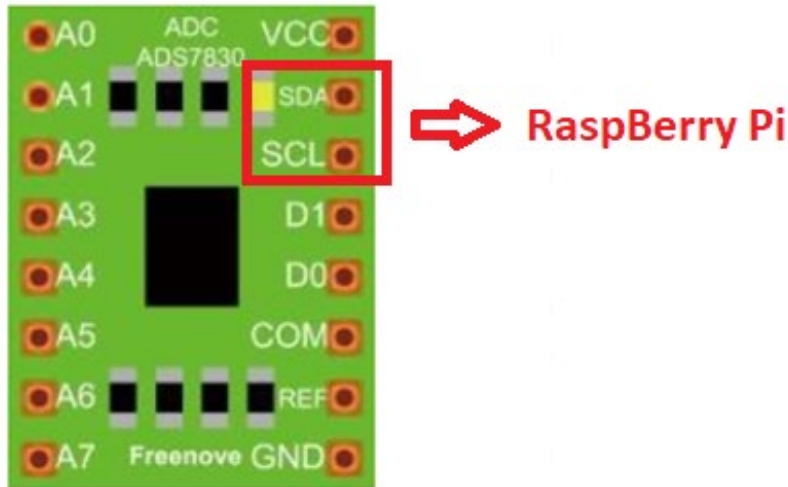


Ce composant n'est pas contrôlé dans votre programme Python. Vous n'avez qu'à le brancher à l'entrée de votre convertisseur (ADS7830).

Circuit intégré ADC (ADS7830): ce circuit permettra de convertir le signal de tension analogique (entre 0 et 3.3 volts) du potentiomètre en une valeur numérique (entre 0 et 255). Ce circuit utilise la technologie I2C, il faut donc activer I2C dans votre Raspberry Pi. Cette technologie permet de relier facilement différents circuits. La connexion I2C est réalisée par l'intermédiaire de deux lignes :

- SDA (Serial Data Line) : ligne de données bidirectionnelle,
- SCL (Serial Clock Line) : ligne d'horloge de synchronisation bidirectionnelle.

Ces deux lignes/broches doivent être reliées aux deux broches I2C correspondantes de votre RaspPi. Ces deux lignes apparaissent encadrées en rouge sur les diagrammes suivants:



Pin#	NAME	NAME	Pin#
01	3.3v DC Power	DC Power 5v	02
03	GPIO02 (SDA1, PC)	DC Power 5v	04
05	GPIO03 (SCL1, PC)	Ground	06
07	GPIO04 (GPIO_GCLK)	(TXD0) GPIO14	08
09	Ground	(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)	(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)	Ground	14
15	GPIO22 (GPIO_GEN3)	(GPIO_GEN4) GPIO23	16
17	3.3v DC Power	(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)	Ground	20
21	GPIO09 (SPI_MISO)	(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)	(SPI_CE0_N) GPIO08	24
25	Ground	(SPI_CE1_N) GPIO07	26
27	ID_SD (PC ID EEPROM)	(PC ID EEPROM) ID_SC	28
29	GPIO05	Ground	30
31	GPIO06	GPIO12	32
33	GPIO13	Ground	34
35	GPIO19	GPIO16	36
37	GPIO26	GPIO20	38
39	Ground	GPIO21	40

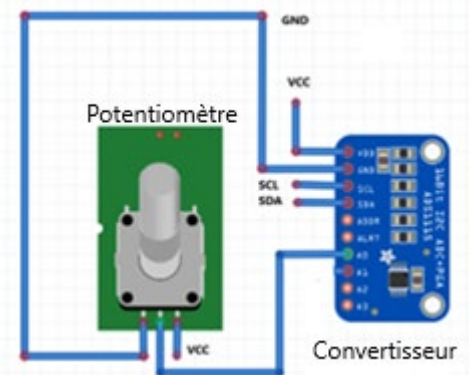
Les autres broches qui nous intéressent sur ce composant sont les suivantes:

- A0 à A7: entrée du signal analogique venant du potentiomètre. Vous n'avez besoin que d'une seule d'entre elles pour votre circuit. Branchez-la à la sortie de votre potentiomètre.
- VCC: 3.3 volts
- GND

Pour faire fonctionner le ADS7830, vous devez importer le module **ADC_Composant**. Ce module est fourni sur Moodle. Voici un circuit de test et un script Python (aussi disponible sur Moodle) pour comprendre le fonctionnement:

```
from ADC_Composant import * # Assurez-vous qu'il soit présent
import RPi.GPIO as GPIO
import time, sys
adc = ADC_Composant() # Déclare un convertisseur
if(adc.detecteI2C(0x4b)): # Le détecte
    adc = ADS7830()
else:
    sys.exit("I2C non connecté sur votre Pi et/ou ADS7830")
# on lit le canal A0 sur le convertisseur
vPot = adc.lectureAnalogique(0)
print(str(vPot))
print("C'est le moment de faire varier votre potentiomètre ...")
time.sleep(3)
vPot = adc.lectureAnalogique(0)
print(str(vPot))
GPIO.cleanup()
```

N.B. Le diagramme montre un convertisseur différent du ADS7830



Indice général: vous n'avez pas besoin de programmation événementielle pour contrôler ce circuit, une boucle suffira pour effectuer des lectures régulières sur le convertisseur.

Remise : vous devrez montrer votre circuit fonctionnel à votre enseignant mardi le 24 octobre et remettre sur LÉA le programme Python ainsi que le schéma conçu sur Fritzing le soir même avant minuit. C'est un travail individuel.